

HOUSE HUNT

Project Submitted to

APSCHE

Bachelor of Technology

In

**ELECTRONICS AND COMMUNICATION
ENGINEERING**

Submitted By

T. G. M. Anusha

Jyothika Surabhathula

Madhuri Chidipi

Nyshika Borra

Sowmya Sri Botta

INTRODUCTION :

HouseHunt is a modern and user-friendly house rental platform that simplifies the process of renting, buying, and selling properties. It offers detailed property listings, advanced search filters, virtual tours, and real-time market insights to help users make informed decisions. Designed for renters, property owners, and real estate professionals, HouseHunt provides a seamless and transparent experience throughout the entire property journey. With 24/7 accessibility and an all-in-one digital marketplace, it makes real estate transactions faster, easier, and more convenient.

DESCRIPTION:

HouseHunt is a powerful and user-centric house rental platform designed to simplify the real estate experience for everyone. It provides an all-in-one solution for renting, buying, and selling properties through a seamless digital marketplace. Users can explore detailed property listings with comprehensive information to make informed decisions. Advanced search filters allow properties to be sorted by location, price, amenities, and other preferences. The platform also offers virtual property tours, enabling users to view homes conveniently from anywhere. Real-time market insights help buyers, renters, and sellers stay updated on current property trends.

HouseHunt connects tenants with landlords and buyers with sellers, making communication and negotiations more efficient. It also streamlines documentation and paperwork, reducing the complexity of property transactions. Real estate agents and brokers can use the platform to market properties, manage clients, and expand their business. By promoting transparency, accessibility, and convenience, HouseHunt creates a trusted environment for all real estate stakeholders. With its innovative features and user-friendly design, HouseHunt transforms the way people discover and manage properties.

It consists of some key features which include:

- 1. Smart Property Search:** Search properties using advanced filters such as location, price range, property type, number of bedrooms, and amenities to quickly find the ideal home.
- 2. Detailed Property Listings:** Access comprehensive property information, including descriptions, high-quality images, pricing, amenities, and contact details for informed decision-making.
- 3. Virtual Property Tours:** Explore homes remotely through virtual tours, allowing users to view properties conveniently before scheduling a physical visit.
- 4. Real-Time Market Insights:** Stay updated with the latest real estate trends, property values, and market information to make better buying, selling, or renting decisions.
- 5. Secure Buyer–Seller & Tenant–Landlord Connection:** Connect directly with property owners, tenants, buyers, sellers, and real estate agents to communicate efficiently and simplify negotiations.

SCENARIO-BASED CASE STUDY

Scenario: HouseHunt

Title: Finding the Perfect Rental Home with HouseHunt

Scenario:

Riya, a software engineer, recently received a job offer in Hyderabad and needed to find a rental apartment before relocating. Since she was unfamiliar with the city and had limited time, visiting multiple properties in person was not practical. She decided to use **HouseHunt** to simplify her search.

Using the app, Riya entered her preferred location, monthly budget, number of bedrooms, and required amenities such as parking, Wi-Fi, and security. Within seconds, HouseHunt displayed a list of verified properties that matched her preferences. She explored detailed property descriptions, viewed high-quality images, and took virtual tours to shortlist the best options.

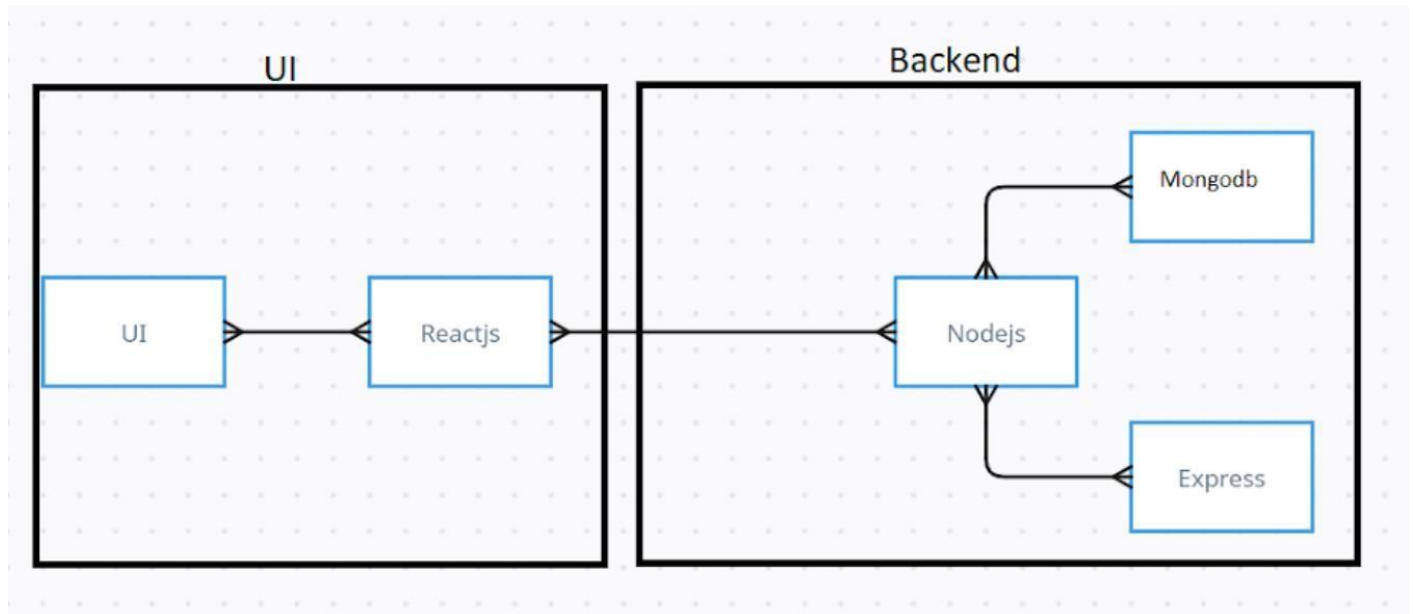
After selecting her preferred apartment, Riya contacted the landlord directly through the platform to discuss rental terms and schedule a final visit. The platform also helped streamline the documentation process, making the rental agreement quick and hassle-free.

Meanwhile, the landlord benefited from HouseHunt by receiving inquiries from genuine tenants and managing property listings efficiently. The real estate agent involved used the platform to communicate with both parties and ensure a smooth transaction.

Outcome:

- Riya found a suitable apartment within a few days.
- The landlord rented the property quickly to a verified tenant.
- The agent completed the transaction efficiently with minimal paperwork.
- HouseHunt provided a transparent, convenient, and reliable experience for all stakeholders, demonstrating how technology can simplify modern real estate transactions.

TECHNICAL ARCHITECTURE



Architecture:

The HouseHunt application follows the **MERN (MongoDB, Express.js, React.js, and Node.js)** architecture, which provides a scalable, efficient, and user-friendly platform for property rental and real estate management. The architecture is divided into two major layers: the **Frontend (UI)** and the **Backend**. The frontend is built using **React.js**, which is responsible for creating an interactive and responsive user interface. It allows users to search properties, apply filters, view property details, register, log in, and communicate with landlords or agents. React.js uses reusable components and communicates with the backend by sending HTTP requests through REST APIs, ensuring a smooth and dynamic user experience without requiring page reloads.

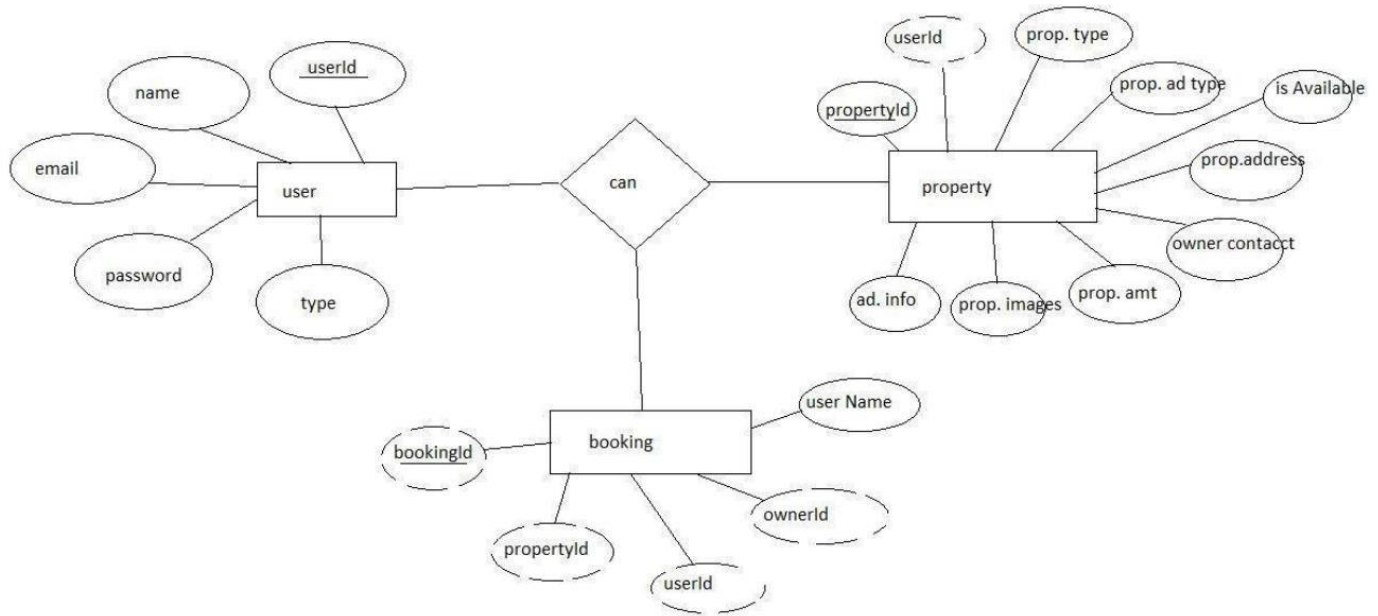
The backend is developed using **Node.js** and **Express.js**, which together handle all server-side operations. **Node.js** acts as the runtime environment that processes client requests, executes business logic, manages user authentication, and communicates with the database. **Express.js** is a lightweight web framework that simplifies API development, routing, middleware integration, and request-response handling. It receives requests from the frontend, validates the data, performs the required operations, and returns the appropriate responses in JSON format.

The application uses **MongoDB** as its database to store and manage all application data, including user accounts, property listings, property images, rental information, agent details, and customer inquiries. MongoDB's document-oriented structure provides flexibility for handling dynamic property information and allows the application to scale efficiently as the number of users and properties grows.

The overall workflow begins when a user performs an action on the HouseHunt interface, such as searching for a property or logging into the system. React.js captures the user's request and sends it to the backend through an API call. Node.js and Express.js process the request, interact with MongoDB to retrieve or update the necessary information, and send the processed data back to the frontend. React.js then dynamically updates the user interface with the received information, allowing users to view results instantly without refreshing the page.

This architecture offers several advantages, including a responsive user interface, fast server-side processing, efficient database management, and clear separation between the frontend and backend components. As a result, HouseHunt provides a secure, reliable, and high-performance platform that simplifies the process of renting, buying, and selling properties while ensuring a seamless experience for users, property owners, and real estate professionals.

ER DIAGRAM



ER Diagram:

The above **Entity Relationship (ER) Diagram** represents the database design of the **HouseHunt** application. It illustrates the relationship between the three main entities: **User**, **Property**, and **Booking**, which together manage the core functionalities of the system. The **User** entity stores information about individuals using the application, including attributes such as **User ID**, **Name**, **Email**, **Password**, and **Type**. The **User ID** serves as the primary key and uniquely identifies each user, while the **Type** attribute distinguishes whether the user is a tenant, property owner, or real estate agent. This entity forms the basis for user authentication and role-based access within the application.

The **Property** entity contains all the details related to properties listed on the platform. It includes attributes such as **Property ID**, **User ID**, **Property Type**, **Property Advertisement Type** (Rent or Sale), **Availability Status**, **Property Address**, **Owner Contact**, **Property Amount**, **Property Images**, and **Advertisement Information**. The **Property ID** acts as the primary key, while the **User ID** serves as a foreign key linking each property to the user who listed it. This relationship allows a single user to own and manage multiple property listings within the application. The property entity stores all the essential information required by potential buyers or tenants to evaluate and select a suitable property.

The **Booking** entity records the transactions and interactions between users and property owners. It contains attributes such as **Booking ID**, **Property ID**, **User ID**, **Owner ID**, and **User Name**. The **Booking ID** uniquely identifies each booking request, while the **Property ID** and **User ID** establish references to the selected property and the user making the booking. The **Owner ID** identifies the owner of the property involved in the transaction. This entity enables the system to maintain booking records, track rental requests, and facilitate communication between tenants and property owners.

The relationship labeled "**can**" connects the **User**, **Property**, and **Booking** entities, indicating that a user can list multiple properties and can also make bookings for available properties. Similarly, each property can receive multiple booking requests from different users over time, creating a one-to-many relationship between properties and bookings. This relationship ensures efficient management of property listings and rental transactions while maintaining data consistency through primary and foreign key constraints.

Overall, this ER diagram provides a structured representation of the HouseHunt database, ensuring that user information, property details, and booking records are logically organized and interconnected. The design supports essential application functionalities such as user registration, property listing, property search, booking management, and owner–tenant interactions. By establishing clear relationships among the entities, the database minimizes data redundancy, improves data integrity, and enables efficient storage and retrieval of real estate information, making the HouseHunt application reliable, scalable, and easy to maintain.

PRE-REQUISITES:

Here are the key prerequisites for developing a full-stack application using Node.js, Express.js, MongoDB, and React.js:

Node.js and npm:

Node.js is a powerful JavaScript runtime environment that allows you to run JavaScript code on the server side. It provides a scalable and efficient platform for building network applications.

Install Node.js and npm on your development machine, as they are required to run JavaScript on the server side.

Download:

Express.js:

Express.js is a fast and minimalist web application framework for Node.js. It simplifies the process of creating robust APIs and web applications, offering features like routing, middleware support, and modular architecture.

Install Express.js, a web application framework for Node.js, which handles server-side routing, middleware, and API development.

Installation: Open your command prompt or terminal and run the following command:

npm install express

MongoDB:

MongoDB is a flexible and scalable NoSQL database that stores data in a JSON-like format. It provides high performance, horizontal scalability, and seamless integration with Node.js, making it ideal for handling large amounts of structured and unstructured data.

Set up a MongoDB database to store your application's data.

Download:

React.js:

React.js is a popular JavaScript library for building user interfaces. It enables developers to create interactive and reusable UI components, making it easier to build dynamic and responsive web applications.

Install React.js, a JavaScript library for building user interfaces.

HTML, CSS, and JavaScript: Basic knowledge of HTML for creating the structure of your app, CSS for styling, and JavaScript for client-side interactivity is essential.

Database Connectivity: Use a MongoDB driver or an Object-Document Mapping (ODM) library like Mongoose to connect your Node.js server with the MongoDB database and perform CRUD (Create, Read, Update, Delete) operations. To Connect the Database with Node JS go through the below provided link:

Front-end Framework: Utilize Reactjs to build the user-facing part of the application, including entering complaints, the status of the complaints, and user interfaces for the admin dashboard.

To make better UI we have also used some libraries like Material UI and Bootstrap.

Version Control: Use Git for version control, enabling collaboration and tracking changes throughout the development process. Platforms like GitHub or Bitbucket can host your repository.

Git: Download and installation instructions can be found at:

Development Environment: Choose a code editor or Integrated Development Environment (IDE) that suits your preferences, such as Visual Studio Code, Sublime Text, or WebStorm.

- Visual Studio Code: Download from <https://code.visualstudio.com/download>

To run the existing Video Conference App project downloaded from GitHub:

Follow the below steps:

Clone the Repository:

- Open your terminal or command prompt.
- Navigate to the directory where you want to store the e-commerce app.

Install Dependencies:

- Navigate into the cloned repository directory:
cd complaint-register
 - Install the required dependencies by running the following commands:
cd frontend
npm install
cd ../backend
npm install
- Start the Development Server:
- To start the development server, execute the following command:
npm start
 - The online complaint registration and management app will be accessible at <http://localhost:3000>

You have successfully installed and set up the online complaint registration and management app on your local machine. You can now proceed with further customization, development, and testing as needed

PROJECT STRUCTURE:

1. Frontend (React.js):

- **src**
 - **components** – Reusable UI components like Navbar, Footer, Login, Signup, Dashboard, etc. **components** – Reusable UI components like Navbar, Footer, Login, Signup, Dashboard, etc.
 - **pages** – Page-level components like HomePage, UserDashboard, AdminPanel, AgentPanel.
 - **routes** – Route definitions for user, admin, and agent navigation.
 - **services** – Axios API calls for interacting with the backend.
 - **App.js** – Root component handling routing and layout.
 - **index.js** – Entry point of the React app.

2. Backend (Node.js + Express.js):

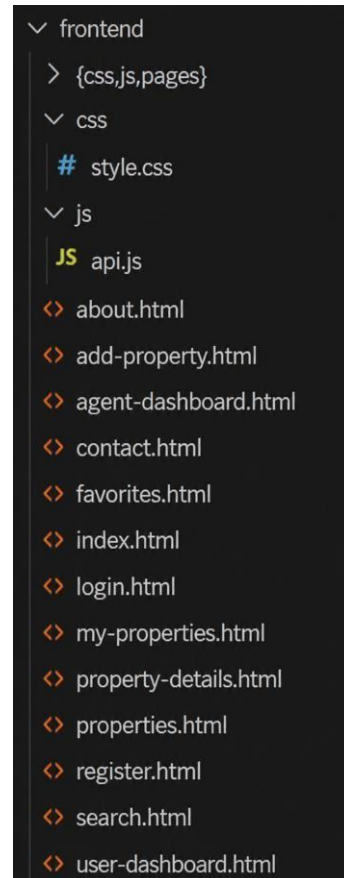
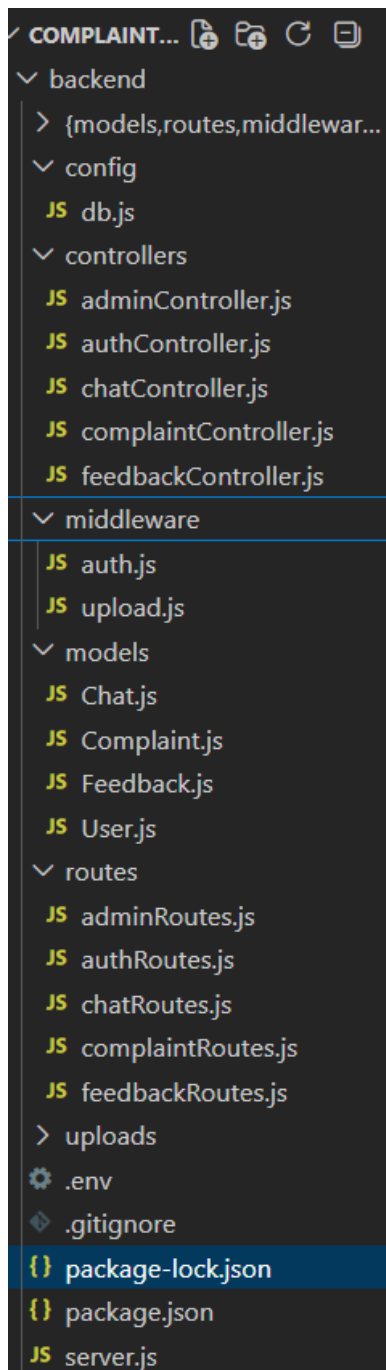
- **controllers** – Contains logic for handling requests (e.g., userController, complaintController).
- **models** – Mongoose schemas for User, Complaint, etc.
- **routes** – Defines all REST API routes (userRoutes, complaintRoutes, adminRoutes).
- **middlewares** – Middleware functions for authentication and role-based access control.
- **server.js** – Main entry point that sets up the Express server and MongoDB connection.

3. Database (MongoDB):

- Stores data in collections such as:
 - **Users** – Stores user/admin/agent information.
 - **Complaints** – Stores complaint details, status, timestamps, and assigned agent.
 - **Sessions/Tokens** – (Optional) For managing login sessions or refresh tokens.

4. Project Flow:

1. **User** signs up or logs in via the frontend.
2. Complaint data is submitted and sent to the backend via **Axios API**.
3. Backend validates and stores the complaint in **MongoDB**.
4. **Admin** can view all complaints and assign them to specific **agents**.
5. **Agents** update the complaint status, which the **user** can track on their dashboard.



The first image is of frontend part which shows all the files and folders that have been used in UI development

The second image is of the Backend part which shows all the files and folders that have been used in the backend development

APPLICATION FLOW:

Online Complaint Registration and Management System

1. Customer/Ordinary User:

- **Role:** Create and manage complaints, interact with agents, and manage profile information.

1. Registration and Login:

- Create an account by providing the necessary information such as email and password.
- Log in using the registered credentials.

2. Complaint Submission:

- Fill out the complaint form with details of the issue, including description, contact information, and relevant attachments.
- Submit the complaint for processing.

3. Status Tracking:

- View the status of submitted complaints in the dashboard or status section.
- Receive real-time updates on the progress of complaints.

4. Interaction with Agents:

- Connect with assigned agents directly using the built-in messaging feature.
- Discuss complaints further and provide additional information or clarification.

5. Profile Management:

- Manage personal profile information, including details and addresses.

2. Agent:

- **Role:** Manage complaints assigned by the admin, communicate with customers, and update complaint statuses.

1. Registration and Login:

- Create an account using email and password.
- Log in using the registered credentials.

2. Complaint Management:

- Access the dashboard to view and manage complaints assigned by the admin.
- Communicate with customers regarding their complaints through the chat window.

3. **Status Update:**

- Change the status of complaints based on resolution or progress.
- Provide updates to customers regarding the status of their complaints.

4. **Customer Interaction:**

- Respond to inquiries, resolve issues, and address feedback from customers.

3. **Admin:**

- **Role:** Oversee the overall operation of the complaint registration platform, manage complaints, users, and agents, and enforce platform policies.

1. **Management and Monitoring:**

- Monitor and moderate all complaints submitted by users.
- Assign complaints to agents based on workload and expertise.

2. **Complaint Assignment:**

- Assign complaints to the desired agents for resolution.
- Ensure timely and efficient handling of complaints.

3. **User and Agent Management:**

- Manage user and agent accounts, including registration, login, and profile information.
- Enforce platform policies, terms of service, and privacy regulations.

4. **Continuous Improvement:**

- Implement measures to improve the platform's functionality, user experience, and security measures.
- Address any issues or concerns raised by users or agents for better service delivery.

Milestone 1:

Project Setup and Configuration:

1. Create project folders and files:

Now, firstly create the folders for frontend and backend to write the respective code and install the essential libraries.

- Client folders.
- Server folders

2. Install required tools and software:

For the backend to function well, we use the libraries mentioned in the prerequisites. Those libraries include

- Node.js.
- MongoDB.
- Bcrypt
- Body-parser

Also, for the frontend we use the libraries such as

- React Js.
- Material UI
- Bootstrap
- Axios

After the installation of all the libraries, the package.json files for the front end look like the one mentioned below.

After the installation of all the libraries, the package.json files for the backend look like the one mentioned below.

```
{ } package.json X
backend > { } package.json
1  {
2    "name": "complaint-management-backend",
3    "version": "1.0.0",
4    "description": "Complaint Management System - Backend (MEN stack)",
5    "main": "server.js",
6    "scripts": {
7      "start": "node server.js",
8      "dev": "nodemon server.js"
9    },
10   "dependencies": {
11     "express": "^4.19.2",
12     "mongoose": "^8.4.0",
13     "bcryptjs": "^2.4.3",
14     "jsonwebtoken": "^9.0.2",
15     "cors": "^2.8.5",
16     "dotenv": "^16.4.5",
17     "express-validator": "^7.1.0",
18     "helmet": "^7.1.0",
19     "multer": "^1.4.5-lts.1",
20     "socket.io": "^4.7.5"
21   },
22   "devDependencies": {
23     "nodemon": "^3.1.0"
24   }
25 }
26
```

Milestone 2:

Backend Development:

- **Set Up Project Structure:**

- Create a new directory for your project and set up a package.json file using npm init command.
- Install necessary dependencies such as Express.js, Mongoose, and other required packages.

- **Set Up Project Structure:**

- Create a new directory for your project and set up a package.json file using npm init command.
- Install necessary dependencies such as Express.js, Mongoose, and other required packages.

- **Create Express.js Server:**

- Set up an Express.js server to handle HTTP requests and serve API endpoints.
- Configure middleware such as body-parser for parsing request bodies and cors for handling cross-origin requests.

- **Define API Routes:**

- Create separate route files for different API functionalities such as authentication, creating, assigning complaints, and chat window.
- Implement route handlers using Express.js to handle requests and interact with the database.

- **Implement Data Models:**

- Create corresponding Mongoose models to interact with the MongoDB database.
- Implement CRUD operations (Create, Read, Update, Delete) for each model to perform database operations.

- **User Authentication:**

- Implement user authentication using strategies like JSON Web

Tokens (JWT).

- Create routes and middleware for user registration, login, and logout.
- Set up authentication middleware to protect routes that require user authentication.

- **Admin Functionality:**

- Implement routes and controllers specific to admin functionalities such as fetching all the data regarding users, complaints, agents.

- **Error Handling:**

- Implement error handling middleware to catch and handle any errors that occur during the API requests.
- Return appropriate error responses with relevant error messages and HTTP status codes.

Milestone 3:

Database Development

1. User Schema:

- The user schema defines the structure of user data stored in the database. It includes fields such as name, email, password, phone, and userType.
- Each user must provide a name, email, password, phone number, and userType (e.g., customer, agent, admin).
- User data is stored in the "user_Schema" collection in the MongoDB database.

2. Complaint Schema:

- The complaint schema specifies the format of complaint data registered by users.
- It contains fields like userId, name, address, city, state, pincode, comment, and status.

- Complaints are associated with users through the `userId` field, and each complaint must have a name, address, city, state, pincode, comment, and status.
- Complaint data is stored in the "complaint_schema" collection in the MongoDB database.

3. Assigned Complaint Schema:

- The assigned complaint schema defines how complaints are assigned to agents for resolution.
- It includes fields such as `agentId`, `complaintId`, `status`, and `agentName`.
- Each assigned complaint is linked to a specific agent (identified by `agentId`) and complaint (identified by `complaintId`).
- The status field indicates the current status of the assigned complaint.
- Assigned complaint data is stored in the "assigned_complaint" collection in the MongoDB database.

4. Chat Window Schema:

- The chat window schema governs the structure of messages exchanged between users and agents regarding specific complaints.
- It comprises fields like name, message, and `complaintId`.
- Messages are associated with a complaint through the `complaintId` field, allowing for easy tracking and retrieval of chat history for each complaint.
- Message data is stored in the "message" collection in the MongoDB database.

Milestone 4:

Frontend Development:

1. Setup React Application:

Bringing Customer Care Registry to life involves a three-step development process. First, a solid foundation is built using React.js. This includes creating the initial application structure, installing necessary libraries, and organizing

the project files for efficient development. Next, the user interface (UI) comes to life. To start the development process for the front end, follow the below steps.

- Install required libraries.
- Create the structure directories.

2.Design UI components:

Reusable components will be created for all the interactive elements you'll see on the screen, from stock listings and charts to buttons and user profiles. Next, we'll implement a layout and styling scheme to define the overall look and feel of the application. This ensures a visually appealing and intuitive interface. Finally, a navigation system will be integrated, allowing you to effortlessly explore different sections of the Customer Care Registry, like making specific complaints or managing your Product complaints.

3.Implement frontend logic:

In the final leg of the front-end development, we'll bridge the gap between the visual interface and the underlying data. It involves the following stages.

- Integration with API endpoints.
- Implement data binding.
-

Milestone 5:

Project Implementation:

On completing the development part, we then ran the application one last time to verify all the functionalities and look for any bugs in it. The user interface of the application looks a bit like the one provided below









